

Entrega Final: Compilador Claudio a Swift

Gramatica ampliada, reglas semanticas, SDT de traduccion, implementacion y pruebas

Curso: Teoria de Compiladores

Autores: Juan David Cardenas Jimenez, Maria Jose Restrepo Ramirez, Jose Zuluaga

Junio de 2026

Convencion visual exigida por el enunciado

Rojo: producciones originales de la gramatica.

Amarillo: reglas semanticas del Quiz 4 con atributos, dominios y acciones.

Azul: acciones SDT que sintetizan codigo Swift.

1. Alcance Del Compilador Final

Este documento presenta el compilador Claudio como resultado final: un lenguaje fuente en español que se analiza en cuatro fases y produce código Swift como lenguaje destino. El punto central no es solo listar componentes, sino mostrar que la traducción se obtiene después de validar correctamente el programa fuente.

La propiedad más importante para la evaluación es la siguiente: Claudio solo muestra código Swift cuando el programa pasa análisis léxico, sintáctico y semántico. Si alguna fase falla, la interfaz explica el error y bloquea la salida destino.

Requisito del enunciado	Evidencia en Claudio
Lexico -> sintactico -> semantico -> SDT	Endpoint /api/compilar encadena las fases y bloquea Swift si alguna falla.
Errores unificados	Cada diagnostico final trae fase, fila, columna, lexema y mensaje.
Tabla de simbolos disponible durante SDT	AnalizadorSemantico construye tabla de simbolos y /api/compilar la retorna junto con la salida.
Tres casos de prueba	Disponibles y seleccionables desde la galeria de programas de la interfaz grafica.
Bonus IA	OpenAI revisa la salida Swift generada y aporta una lectura complementaria sin reemplazar al compilador.

Cadena de fases:

```
Fuente Claudio
-> Lexer: tokens + errores lexicos
-> Parser RD: arbol + errores sintacticos
-> Analizador semantico: tabla de simbolos + SEM-1..SEM-7
-> SDT Swift: codigo destino + mapeo linea a linea
-> IA opcional: revision del Swift generado
```

2. Gramatica Base y SDT

Las producciones están escritas en forma BNF compacta. Los no terminales con repetición usan recursividad derecha o epsilon; sus acciones SDT concatenan fragmentos Swift en orden de aparición.

2.1 Programa y declaraciones

```
programa -> declaracion programa | ε
declaracion -> importacion | decl_variable | def_funcion | def_clase | sentencia
importacion -> importar ID
```

SDT-PROG

programa.swift = declaracion.swift + "\n" + programa1.swift.

programa -> ε sintetiza cadena vacía; no altera la salida.

importacion.swift = "import " + ID.lexema.

2.2 Variables y tipos

```
decl_variable -> (var | sea) tipo ID = expresion
tipo -> entero | real | cadena | booleano | ID
tipo_basico -> entero | real | cadena | booleano
```

SEM-1 y SEM-4: declaracion de variable

Atributos: ID.nombre, tipo.declarado, expresion.tipo, ambito.actual, tabla_simbolos

Dominio / restriccion: ID no debe existir en el ambito actual; expresion.tipo debe ser compatible con tipo.declarado. entero puede promoverse a real.

Accion: Reportar duplicado o incompatibilidad; si no hay duplicado, registrar simbolo con tipo e inmutabilidad.

SDT-VAR

Si el declarador es var: decl_variable.swift = "var ID: TipoSwift = expr.swift".

Si el declarador es sea: decl_variable.swift = "let ID: TipoSwift = expr.swift".

TipoSwift(entero, real, cadena, booleano) = Int, Double, String, Bool.

2.3 Funciones y parametros

```
def_funcion -> funcion tipo_ret ID ( parametros ) hacer bloque fin_funcion
tipo_ret -> tipo_basico | ε
parametros -> param_lista | ε
param_lista -> tipo ID param_resto
param_resto -> , tipo ID param_resto | ε
sent_retornar -> retornar expresion
```

SEM-FUNC: ambito y parametros

Atributos: funcion.nombre, tipo_ret, parametros.lista, ambito.funcion

Dominio / restriccion: El nombre de funcion no debe duplicarse en el ambito actual; cada parametro se declara en el ambito de la funcion.

Accion: Registrar la funcion como simbolo, abrir ambito, registrar parametros, analizar bloque y cerrar ambito.

SDT-FUNC

def_funcion.swift = "func ID(params.swift) -> TipoRet {\n" + bloque.swift + "\n}".

Si tipo_ret es ε, se omite "-> TipoRet".

param_lista.swift = "_ ID: TipoSwift" + param_resto.swift; param_resto epsilon sintetiza cadena vacia.

sent_retornar.swift = "return " + expresion.swift.

2.4 Clases, atributos y metodos

```
def_clase -> clase ID herencia_opt hacer cuerpo_clase fin_clase
herencia_opt -> hereda ID | ε
cuerpo_clase -> miembro_clase cuerpo_clase | ε
miembro_clase -> def_atributo | def_metodo
def_atributo -> atributo tipo ID
def_metodo -> metodo ID ( parametros ) hacer bloque fin_funcion
```

SEM-CLASE: nombres de clase, atributos y metodos

Atributos: clase.nombre, atributo.nombre, metodo.nombre, ambito.clase

Dominio / restriccion: Cada nombre declarado dentro de la clase debe ser unico en el ambito de clase.

Accion: Registrar clase, abrir ambito de clase, registrar atributos/metodos y analizar cuerpos de metodo.

SDT-CLASS

def_clase.swift = "class ID[: Super] {\n" + cuerpo.swift + "\n}".

def_atributo.swift = "var ID: TipoSwift = default(TipoSwift)" para evitar propiedades almacenadas sin valor inicial.

default(Int, Double, String, Bool) = 0, 0.0, "", false.

def_metodo.swift = "func ID(params.swift) {\n" + bloque.swift + "\n}".

2.5 Bloques y sentencias

```
bloque -> sentencia bloque' bloque' -> sentencia bloque' | ε
sentencia -> sent_id | sent_si | sent_para | sent_mientras
| sent_retornar | sent_imprimir | sent_romper | sent_continuar | decl_variable
sent_id -> (ID | este) resto_id
resto_id -> = expresion | . ID resto_id | ( argumentos ) | ε
argumentos -> arg_lista | ε
arg_lista -> expresion argresto
argresto -> , expresion argresto | ε
```

SEM-2, SEM-3 y SEM-5: uso y asignacion

Atributos: ID.nombre, ID.tipo, ID.inmutable, expresion.tipo, tabla_simbolos

Dominio / restriccion: El identificador debe existir; una constante declarada con sea no puede reasignarse; el tipo asignado debe ser compatible.

Accion: Reportar identificador no declarado, reasignacion de constante o incompatibilidad de tipo.

SDT-STMT

sent_id asignacion: "lhs.swift = expresion.swift"; este.campo se traduce a self.campo.

sent_id llamada: "ID(args.swift)"; acceso por punto conserva el receptor y traduce este a self.

argumentos y restos con epsilon sintetizan cadena vacia; las repeticiones agregan coma y expresion.swift.

sent_imprimir.swift = "print(" + expresion.swift + ")"; romper/continuar -> break/continue.

2.6 Condicionales y ciclos

```
sent_si -> si expresion entonces bloque rama_sino fin_si
rama_sino -> sino bloque | ε
sent_para -> para ID desde expresion hasta expresion paso_opt hacer bloque fin_para
paso_opt -> paso expresion | ε
sent_mientras -> mientras expresion hacer bloque fin_mientras
```

SEM-6 y SEM-7: condiciones y limites

Atributos: condicion.tipo, desde.tipo, hasta.tipo, paso.tipo

Dominio / restriccion: La condicion de si/mientras debe ser booleana; desde, hasta y paso de para deben ser enteros o reales.

Accion: Reportar condicion no booleana o limites/paso no numericos. La variable de para se declara como entero en su ambito.

SDT-CONTROL

sent_si.swift = "if cond.swift {\n" + bloque.swift + "\n}" + rama_sino.swift.

rama_sino epsilon sintetiza cadena vacia; rama_sino con sino sintetiza " else {\n bloque.swift \n}".

sent_para.swift = "for ID in stride(from: desde.swift, through: hasta.swift, by: paso.swift) {\n bloque.swift \n}".

paso_opt epsilon sintetiza 1; sent_mientras.swift = "while cond.swift {\n bloque.swift \n}".

2.7 Expresiones

```

expresion -> expr_or
expr_or -> expr_and expr_or' expr_or' -> o expr_and expr_or' | ε
expr_and -> expr_rel expr_and' expr_and' -> y expr_rel expr_and' | ε
expr_rel -> expr_add expr_rel' expr_rel' -> op_rel expr_add | ε
expr_add -> expr_mul expr_add' expr_add' -> + expr_mul expr_add' | - expr_mul expr_add' | ε
expr_mul -> expr_pot expr_mul' expr_mul' -> * expr_pot expr_mul' | / expr_pot expr_mul' | % expr_pot
expr_mul' | ε
expr_pot -> expr_unaria expr_pot' expr_pot' -> ** expr_unaria expr_pot' | ε
expr_unaria -> no expr_unaria | - expr_unaria | expr_primaria
expr_primaria -> literal | verdadero | falso | nulo | nuevo ID(argumentos) | este sufijo_id | ID sufijo_id |
( expresion )
sufijo_id -> ( argumentos ) | . ID sufijo_id | ε

```

SDT-EXPR

Operadores aritmeticos y relacionales conservan su simbolo Swift: +, -, *, /, %, ==, !=, <, >, <=, >=.

y -> &&, o -> ||, no -> !. verdadero/falso/nulo -> true/false/nil.

nuevo ID(args) -> ID(args). este.sufijo -> self.sufijo.

a ** b -> pow(Double(a), Double(b)); los nodos epsilon de expresion sintetizan cadena vacia.

3. Reglas Semanticas Implementadas

Regla	Logica tecnica	Como se aplica
SEM-1	Evita nombres repetidos dentro del mismo ambito.	Antes de registrar una variable, funcion, clase, atributo o parametro, se consulta el ambito actual de la tabla de simbolos.
SEM-2	Impide usar identificadores inexistentes.	Cada aparicion de un identificador en expresiones, llamadas o asignaciones se resuelve contra la pila de ambitos.
SEM-3	Protege constantes declaradas con sea.	La tabla conserva si el simbolo es inmutable; una asignacion posterior a ese nombre se reporta como error.
SEM-4	Valida que el valor inicial coincida con el tipo declarado.	Se infiere el tipo de la expresion y se compara con el tipo de la declaracion, permitiendo entero hacia real.
SEM-5	Valida asignaciones posteriores.	El tipo almacenado en la tabla se contrasta con el tipo inferido de la nueva expresion asignada.
SEM-6	Exige condiciones booleanas.	Las expresiones de si y mientras deben inferirse como booleano; un entero, cadena o real no sirve como condicion.
SEM-7	Exige limites numericos en para.	desde, hasta y paso se infieren y deben pertenecer al dominio numerico: entero o real.

La tabla de simbolos almacena nombre, tipo, inmutabilidad, estado de inicializacion, ambito, fila y columna. La inferencia de tipos es conservadora: si una expresion no se puede determinar con certeza, se evita un falso positivo semantico.

4. Como Se Materializa En Claudio

La solucion se implemento como una cadena de fases conectadas. Cada fase recibe el resultado de la anterior, agrega informacion propia y decide si el proceso puede continuar. Por eso la salida Swift no aparece como una conversion directa de texto, sino como el ultimo paso de una validacion completa del programa fuente.

En la interfaz, esta logica se observa de forma progresiva: primero se reconocen tokens, luego se construye el arbol sintactico, despues se valida la tabla de simbolos y finalmente se muestra el codigo Swift solo si no existen errores pendientes.

Fase	Entrada	Proceso tecnico	Salida
Lexica	Texto Claudio	Recorre caracteres, clasifica lexemas y conserva fila/columna.	Tokens, errores lexicos y simbolos lexicos.
Sintactica	Tokens validos	Aplica la gramatica descendente recursiva, construye AST y recupera errores cuando es posible.	Arbol de derivacion, diagnosticos sintacticos y estado valido/invalido.
Semantica	AST	Recorre el arbol con tabla de simbolos por ambitos e inferencia conservadora de tipos.	Errores SEM-1..SEM-7 y tabla de simbolos.
SDT Swift	Programa semanticamente valido	Aplica acciones de traduccion asociadas a producciones: tipos, bloques, condiciones, ciclos, funciones y expresiones.	Codigo Swift y mapeo Claudio->Swift.
IA opcional	Swift generado	OpenAI revisa la salida destino sin reemplazar las decisiones deterministicas del compilador.	Resumen de validacion y sugerencias si aplica.

Compuerta de generacion Swift

Situacion	Comportamiento implementado
Hay error lexico	Se reporta la fase lexica con posicion y lexema; no se ejecuta la traduccion a Swift.
Hay error sintactico	Se reporta el token encontrado y el fragmento esperado por la gramatica; no se genera codigo destino.
Hay error semantico	Se reporta la regla SEM correspondiente y la tabla de simbolos parcial; Swift queda bloqueado.
No hay errores	Se sintetiza Swift, se muestra el mapeo fuente-destino y se activa la validacion IA opcional.

5. Pruebas Desde La Interfaz Grafica

Los tres casos exigidos se demuestran desde la UI de Claudio, usando el selector de programas de ejemplo y el metodo Semantico. Esto permite al evaluador observar el comportamiento real del compilador sin ejecutar comandos.

Caso en la UI	Que demuestra	Observacion esperada
Final. Valido con Swift	Programa correcto con funcion, constante, ciclo para, acumulador, imprimir y condicional.	La pestaña Swift muestra codigo destino, mapeo Claudio->Swift y validacion IA.
Final. Error semantico sin Swift	Sintaxis correcta con SEM-3, SEM-4 y SEM-6.	La pestaña Swift muestra bloqueo; los errores aparecen como semanticos.
Final. Error lexico/sintactico sin Swift	Falta entonces, parentesis sin cerrar y caracter @.	La pestaña Swift muestra bloqueo; los errores aparecen como lexicos y sintacticos.

Las capturas siguientes fueron tomadas desde la aplicacion desplegada en claudio.dautia.com. En cada una se observa el estado de la cadena de fases y el comportamiento de la pestaña Swift.

5.1 Caso Valido: Swift Generado

C

Claudio

Lexico

() Desc. Recursivo

Predictivo LL(1)

Semantico

Final. Valido con Swift

Analizar

Ctrl+Enter

✓Codigo

✓Lexico

✓Sintactico

✓Semantico

Swift

Reglas semanticas sobre el AST — tipos, ambitos, constantes y tabla de simbolos

61 tokens

✓ valida 399 nodos

✓ sem. valida 6 simbolos

1 // Entrega final: debe pasar lexico, sintactico y semantico, y generar Swift

2 function entero doble(entero n) hacer

3 retornar n * 2

4 fin_funcion

5

6 sea entero LIMITE = 4

7 var entero acumulado = 0

8

9 para i desde 1 hasta LIMITE paso 1 hacer

10 var entero valor = doble(i)

11 acumulado = acumulado + valor

12 imprimir(valor)

13 fin_para

14

15 si acumulado > 0 entonces

16 imprimir(acumulado)

17 fin_si

Tokens 61

Tabla Simbolos 6

Swift

Errores

Swift generado 17 lineas · 14 efectivas · 6 simbolos · 17 mapeos

Copiar

Swift

CLAUDIO	SWIFT	MAPEO
1 // Entrega final: debe pasar	1 // Entrega final: debe pasar	// Entrega final: debe pasar lexico, sintactico y semantico, y generar Swift
2 function entero doble(entero n) hacer	2 func doble(_ n: Int) → Int {	// Entrega final: debe pasar lexico, sintactico y semantico, y generar Swift
3 retornar n * 2	3 return n * 2	
4 fin_funcion	4 }	funcion entero doble(entero n) hacer
5	5	func doble(_ n: Int) → Int {
6 sea entero LIMITE = 4	6 let LIMITE: Int = 4	retornar n * 2
7 var entero acumulado = 0	7 var acumulado: Int = 0	return n * 2
8	8	
9 para i desde 1 hasta LIMITE ;	9 for i in stride(from: 1, through: LIMITE, by: 1) {	fin_funcion
10 var entero valor = doble(i)	10 var valor: Int = doble(i)	
11 acumulado = acumulado + valor	11 acumulado = acumulado + valor	
12 imprimir(valor)	12 print(valor)	
13 fin_para	13 }	
14	14	sea entero LIMITE = 4
15 si acumulado > 0 entonces	15 if acumulado > 0 {	let LIMITE: Int = 4
16 imprimir(acumulado)	16 print(acumulado)	var entero acumulado = 0
17 fin_si	17 }	var acumulado: Int = 0

Gramatica BNF

Validacion IA Swift

La traducción a Swift es sintácticamente razonable y mantiene la intención del programa Claudio. Las estructuras están bien balanceadas y el flujo lógico se conserva.

lista

Caso Final. Valido con Swift: la interfaz confirma fases validas y muestra codigo Swift, mapeo y validacion IA.

5.2 Caso Semantico: Swift Bloqueado

The screenshot shows the Claudio compiler interface with the following components:

- Top Bar:** Claudio, Lexico, Desc. Recursivo, Predictivo LL(1), Semantico, Final. Error semantico sin Swift, Analizar (Ctrl+Enter).
- Code Editor:**

```
1 // Entrega final: la sintaxis es correcta, pero la
  semantica bloquea Swift
2 sea entero limite = 3
3 limite = 7
4
5 var entero total = "mucho"
6
7 si total entonces
8     imprimir(total)
9 fin_si
```
- Right Panel:**
 - Tokens 22, Tabla Simbolos 2, Swift, Errores 3.
 - Swift bloqueado:** corrige las fases anteriores para generar salida destino.
 - semantic: 3**
 - SEM-3 limite:** No se puede reasignar la constante 'limite' (declarada con 'sea' en fila 2).
 - SEM-4 total:** Tipo incompatible en la declaración de 'total': se declaró 'entero' pero el valor asignado es de tipo 'cadena'.
 - SEM-6 si:** La condición del 'si' es de tipo 'entero', pero debe ser de tipo 'booleano'.
- Bottom Left:** Gramatica BNF button.

Caso Final. Error semantico sin Swift: la sintaxis es aceptada, pero los errores SEM-3, SEM-4 y SEM-6 bloquean la salida destino.

5.3 Caso Lexico/Sintactico: Swift Bloqueado

The screenshot shows the Claudio compiler interface with the following components:

- Top Bar:** Claudio, Lexico, Desc. Recursivo, Predictivo LL(1), Semantico, Final. Error lexico/sintactico sin Swift, Analizar (Ctrl+Enter).
- Code Editor:**

```
1 // Entrega final: errores tempranos, no debe generarse
  Swift
2 var entero x = 10
3
4 si x > 5
5     imprimir("Mayor"
6 fin_si
7
8 imprimir(@)
```
- Right Panel:**
 - Tokens 18, Tabla Simbolos, Swift, Errores 4.
 - Swift bloqueado:** corrige las fases anteriores para generar salida destino.
 - lexico: 1**
 - sintactico: 3**
 - lexico 8:10 @:** Caracter no reconocido '@' en fila 8, col 10.
 - sintactico 5:5 imprimir:** Token inesperado 'imprimir' en condicional si. Esperado: entonces.
 - sintactico 6:1 fin_si:** Token inesperado 'fin_si' en imprimir. Esperado:).
 - sintactico 8:11):** Token inesperado ')' en argumento de imprimir. Esperado: -, ID, NUM_ENTERO, no, verdadero, nuevo, f, falso, nulo, CADENA_LIT, NUM_REAL, este.
- Bottom Left:** Gramatica BNF button.

Caso Final. Error lexico/sintactico sin Swift: el compilador detecta errores tempranos y no permite generar codigo destino.

5.4 Respaldo Tecnico De Las Pruebas

Los mismos programas usados en la UI tambien quedan guardados como archivos fuente para reproducibilidad y regresion automatizada.

Archivo fuente	Caso UI equivalente
tests/final/caso_valido.claudio	Final. Valido con Swift
tests/final/caso_semantico.claudio	Final. Error semantico sin Swift
tests/final/caso_lexico_sintactico.claudio	Final. Error lexico/sintactico sin Swift

Verificacion automatizada complementaria:

```
cd backend
python3 -m unittest test_quiz3.py test_quiz3_diagnostic_cases.py test_quiz4_semantico.py test_entrega_final.py

cd ../frontend
npm run lint
npm run build
```

6. Bonus IA Con OpenAI

La integracion con OpenAI se usa como una revision posterior del lenguaje destino. Primero Claudio decide, con sus reglas propias, si el programa fuente es valido. Solo cuando esa validacion termina sin errores se envia el Swift generado a revision complementaria.

Esto evita que la IA sustituya al compilador: OpenAI no acepta ni rechaza programas Claudio. Su papel es revisar la calidad de la traduccion generada y explicar si el Swift conserva la intencion del programa fuente.

Aspecto	Descripcion para la entrega
Momento de uso	Despues de lexico, sintactico, semantico y SDT. Si hay errores, no se consulta IA porque no existe salida destino valida.
Informacion revisada	Codigo Claudio original y codigo Swift generado por las acciones de traduccion.
Criterios enviados	Sintaxis Swift razonable, llaves y bloques balanceados, operadores y literales traducidos, y conservacion de la intencion del programa.
Resultado mostrado	Resumen breve en la interfaz, junto con posibles problemas y sugerencias sobre el Swift.
Comportamiento si no esta disponible	La compilacion deterministica se conserva; simplemente no se muestra la revision complementaria.

En la prueba valida de la interfaz se observa esta integracion: la pestaña Swift muestra el codigo destino y, debajo, un resumen de validacion IA que indica si la traduccion conserva la estructura e intencion del programa Claudio.

7. Conclusiones

- La entrega final queda cerrada como un flujo de compilacion completo: las fases no son pantallas aisladas, sino una cadena con compuertas claras.
- La gramatica, las reglas semanticas y las acciones SDT se mantienen alineadas con la implementacion real: tipos, bloques, ciclos, funciones, clases y expresiones tienen traduccion definida.
- La visualizacion Swift facilita defender el resultado: muestra salida destino, mapeo fuente-destino, estado de validacion y bloqueo explicito cuando hay errores.
- OpenAI se conserva como bonus educativo y no como dependencia critica: el compilador sigue funcionando aunque la IA no este disponible.